1501116 & 15011001 Computer Programming

Chapter 8 Methods

Academic Year 2026/01

School of Applied Digital Technology, Mae Fah Luang University

Revised version by: *Patcharaporn Panwong*
Paweena Suebsombut
Sujitra Arwatchananukul

Original version by: *Surapong Uttama*

Lesson Plan

Week	Weekly Summary
Week 10 (5-9 Oct 20206)	**Chapter 7:** Iteration Statement: Loop Part 2
Week 11 (12-16 Oct 2026)	**Chapter 8:** Methods
Week 12 (19-23 Oct 2026)	**Chapter 9:** Array
Week 13 (26-30 Oct 2026)	**Chapter 10:** Class and Object
Week 14 (2-6 Nov 2026)	Group Project Implementation
Week 15 (9-13 Nov 2026)	**Group Project Presentation** ★★★
Week 16 (16-20 Nov 2026)	**Group Project Presentation** ★★★
Final Exam (Thursday 26 November 2026, 09:00 - 12:00)	

FIRST SEMESTER 2026

The Online Teaching Evaluation System

NOW OPEN . . .

You can complete the online teaching evaluation for all subjects from now until November 22, 2026

Reminders

The students who do not complete the online teaching evaluation before November 8, 2026.
"The REG system will be locked."

Scan to Evaluate

Contact Info

Division of Quality Assurance and Curriculum Development.
AS Building, 4th Floor
E-mail : qa.eval@mfu.ac.th
Tel. 053 916 407



Link:
https://evaloutcome.mfu.ac.th/login

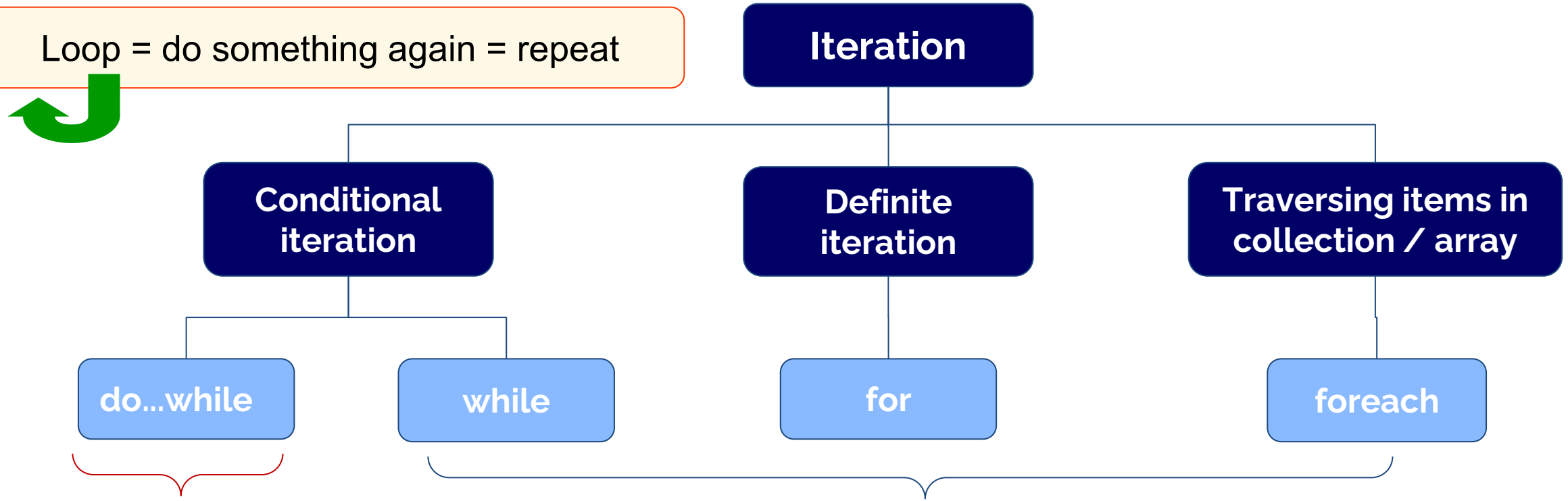
Outline

- Review
- What is Method?
  - Defining method
  - Calling / Invoking Method
  - Types of Method
- Built-in / Standard C# Methods / Predefined Methods
- User-defined Methods
- Summary

REVIEW

Kinds of Iteration Statements

Loop = do something again = repeat



Posttest Loop

```
do
{

} while ( condition )
```

Pretest Loop

```
while ( condition )
{

}
```

```
for ( initialization; condition; updation)
{

}
```

```
foreach ( Data_Type  variable_name  in Collection/array )
{

}
```

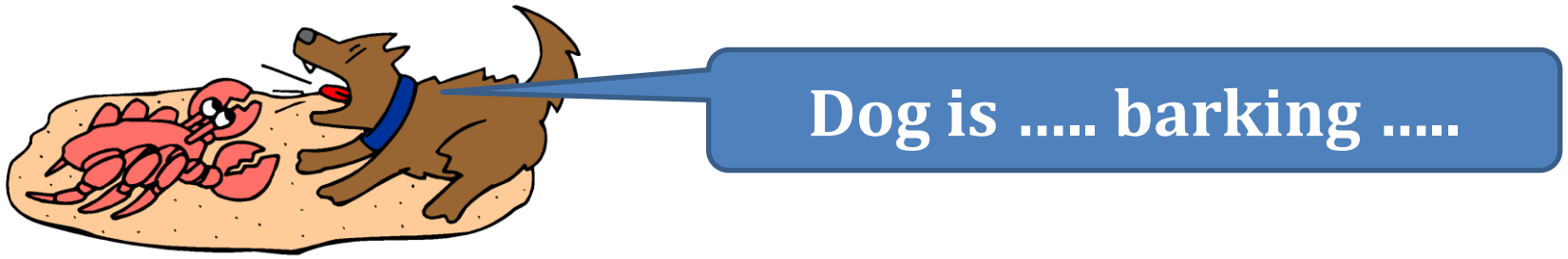
Self study: collection in C#

WHAT IS METHOD?

Methods:

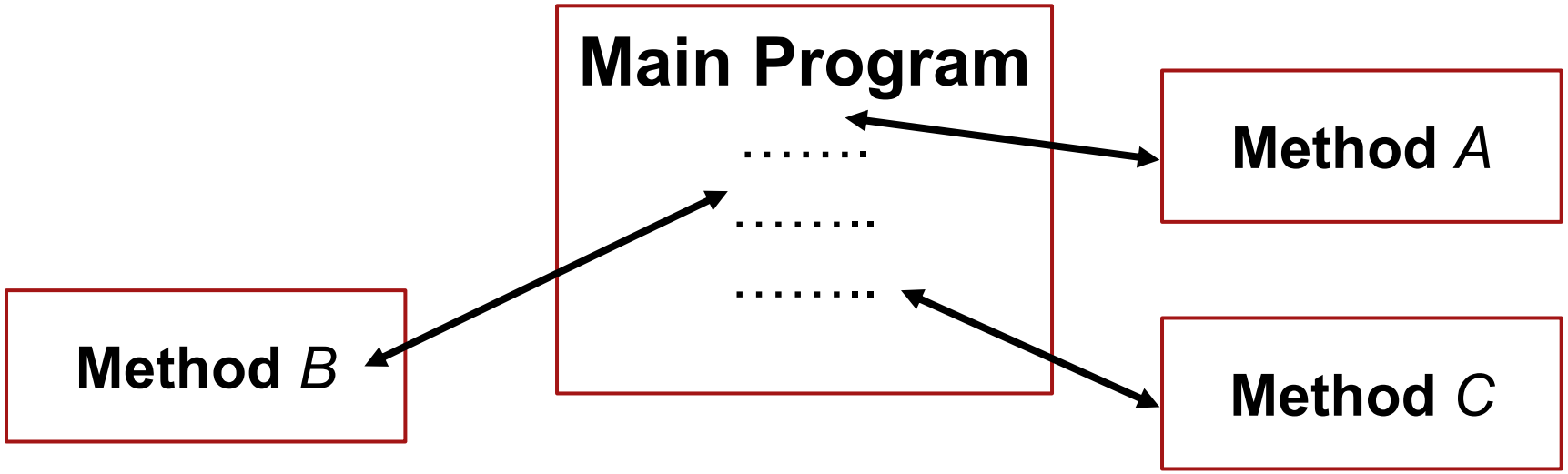
- **Method = a group of statements for a specific purpose**
  - separate a program to smaller parts
  - make a program easier to read and understand
  - support a code reuse

- **A method is an action or behavior of class/program**
  - name of method should be "verb"

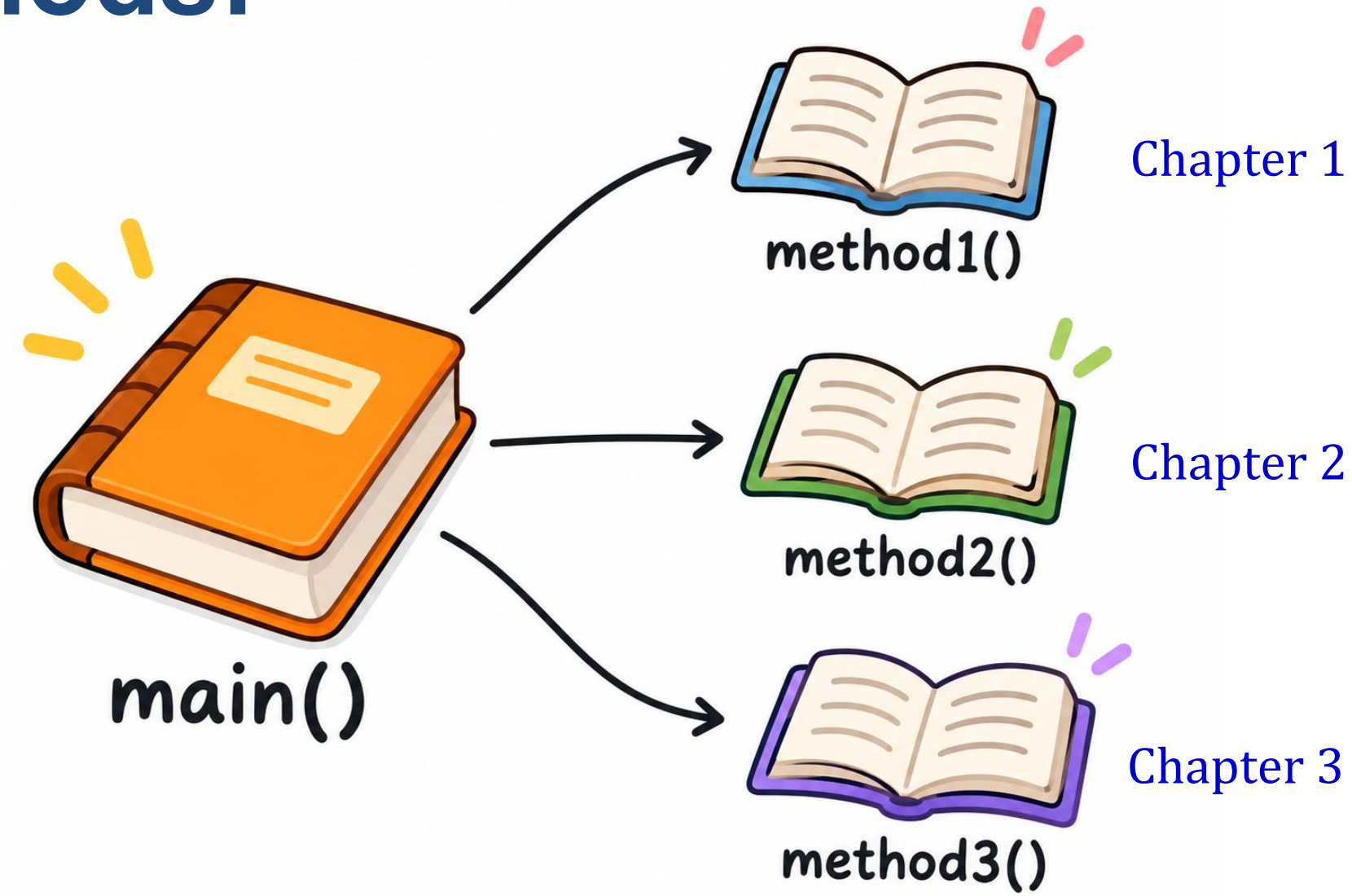


Methods:

- Method is a small sub-routine with its **input**, **output** and **logic.**
- Cut a sequence of statements which you often use, and make it as method



Methods:



Basic Components of Method

Syntax

```
Modifier return_type method_name (parameters)
{
        Method body
}
```

1. **Modifier:** public, private and etc.
2. **Return type:** int, float and etc. OR void

Not returning any value

any data types

Basic Components of Method (Cont.)

Syntax

```
Modifier return_type method_name (parameters)
{
        Method body
}
```

3. **Method name:** //action name

Recommend: Start with uppercase letter for each word e.g., SayHi()

4. **Parameter(s):** //declaring variables used to receive the value

5. **Method body:** {Method's code}

Defining Method

Syntax

```
Modifier return_type method_name (parameters)
{
        Method body
}
```



*** Our methods must be outside of the Main( ).***



Example

```
public static void SayHi (string name)
{
        Console.WriteLine("Hello, "+ name);
}
```

Calling / Invoking Method

Explanation:

Inside **Main()** → call the **method**: **SayHi()**

```
using System;
public class Program
{
    public static void Main(){
        SayHi("Teddy");
    }
    public static void SayHi(string name){
        Console.WriteLine("Hello,   " + name);
    }
}
```

Main method

SayHi method

Calling + Passing value to method

Output / Result

```
Hello,   Teddy
```

- ***static method*** *can …*
  - *Be called without creating an object*
  - *Only call other static methods*

Types of Method

- **By Source**
  1. Built-in / Standard C# Methods / Predefined method
  2. User-defined Methods

Example: WriteLine() from Console class

Example: SayHi() from the previous slide

- **By Parameter and Return Data**
  1. **No** parameter + **No** return data
  2. **No** parameter + **Return** data
  3. Parameter(s) + **No** return data
  4. Parameter(s) + **Return** data

Type of Method: By Source

BUILT-IN / STANDARD C# METHODS / PREDEFINED METHODS

Built-in / Standard C# Methods / Predefined Methods

- Ready to use
- Frequently-used methods
  - Input / Output methods from Console Class
    - WriteLine();
    - ReadLine();
  - Math methods from Math Class
    - Math.Pow(x,n);
  - Etc.



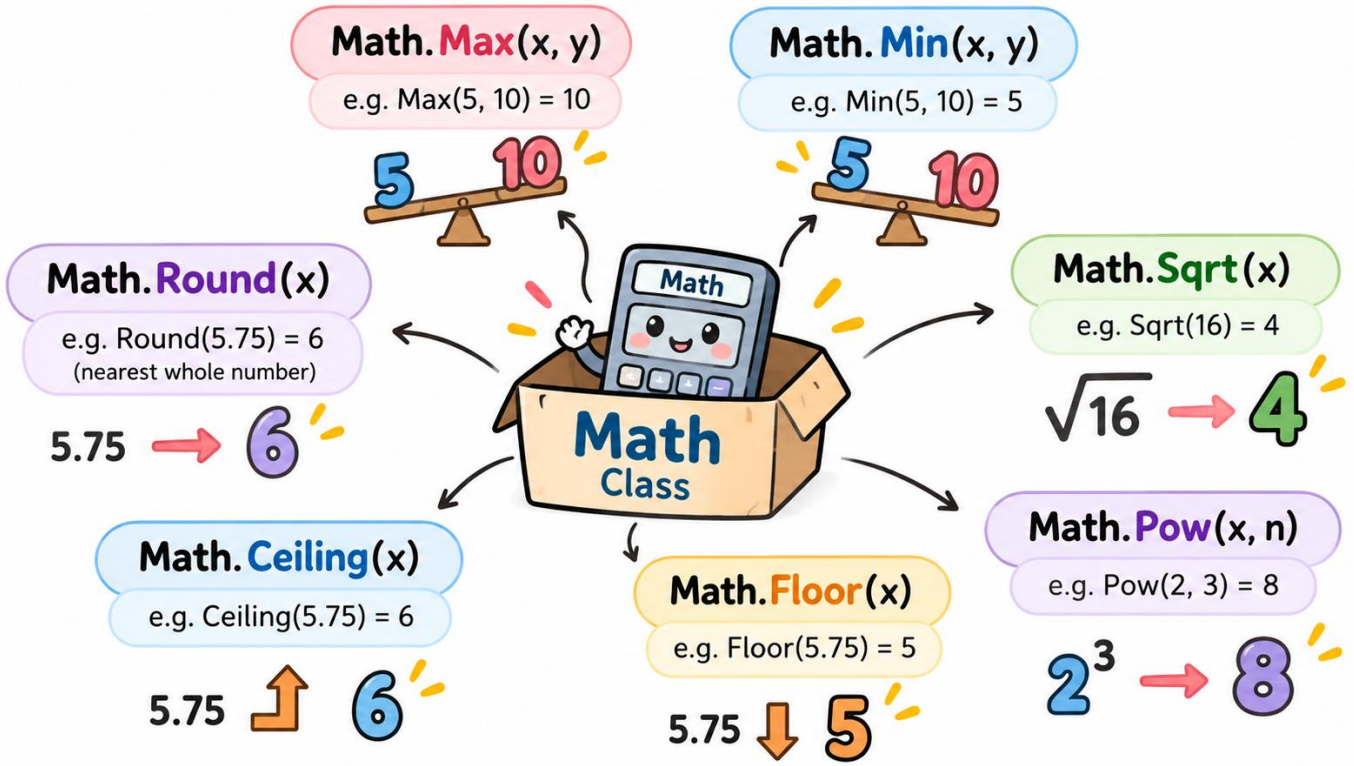
Math.Pow(2, 10) //2 raised to 10th power

Math Methods

- General **math methods** are in **Math Class**

Examples:

- Math.Max(x,y)
- Math.Min(x,y)
- Math.Sqrt(x)
- Math.Pow(x,n)
- Math.Round(x) //return a number to the nearest whole number
- Math.Ceiling(x) //e.g., Ceiling 5.75 = 6
- Math.Floor(x) //e.g., Floor 5.75 = 5



More methods in Match class: Microsoft

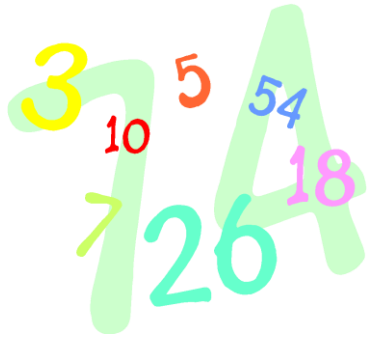
Example#1

```
double num1=5.75, num2=10.2;
Console.WriteLine("Number1 = {0}, Number2 = {1}", num1, num2);
Console.WriteLine("Max = " + Math.Max(num1, num2)); //Find the highest value
Console.WriteLine("Min = " + Math.Min(num1, num2)); //Find the lowest value
//Round a number to the nearest whole number
Console.WriteLine("Round {0}  = {1}", num1, Math.Round(num1));
Console.WriteLine("Round {0}  = {1}\n", num2, Math.Round(num2));
Console.WriteLine("Ceiling {0}  = {1}", num1, Math.Ceiling(num1));
Console.WriteLine("Floor {0}  = {1}", num1, Math.Floor(num1));
Console.WriteLine("Square root of 4 = " + Math.Sqrt(4));
```

Output / Result

```
Number1 = 5.75, Number2 = 10.2
Max = 10.2
Min = 5.75
Round 5.75  = 6
Round 10.2  = 10

Ceiling 5.75  = 6
Floor 5.75  = 5
Square root of 4 = 2
```



More methods in Match class: Microsoft

Be Careful!

Check a method's parameters carefully

1 Method Signatures

Math.Max has multiple versions with different parameter types.

```
Max(int, int)
Max(double, double)
Max(decimal, decimal)
```

Same method name, different parameter types.

2 Parameters

Math.Max needs 2 input values (parameters).

```
Math.Max(x, y)
```

x	y
10	2
First parameter (input value)	Second parameter (input value)

3 Result

Math.Max returns the larger value of the two inputs.

```
Math.Max(10, 2)
→ 10
```

Returns the larger value.

Remember

1. Check the number of parameters.
2. Check the data type of the parameters.
3. Check what the method returns.

Type of Method: By Source

USER-DEFINED METHODS

4 Patterns of User-defined Method

Methods can be classified by 2 things: **Parameters** and **Return Data**

1 No parameter + No return data

(no input) → Method → (no output)

```
void ShowMenu()
```

2 No parameter + Return data

(no input) → Method → return value

```
int GetChoice()
```

3 Parameter(s) + No return data

input value(s) → Method → (no output)

```
void PrintResult(int score)
```

4 Parameter(s) + Return data

input value(s) → Method → return value

```
double Add(double x, double y)
```

1. No parameter + No return data



1. No parameter + No return data

```
Modifier return_type method_name (parameters)
{
        statements;
        return variable or data; //or nothing
}
```



```
public static void method_name (  )
{
        statements;
        return  ;  //or nothing
}
```

No parameter

No return data

1. No parameter + No return data

```
public static void method_name()
{
        statements;
        return;
}
```



```
public static void method_name( )
{
        statements;
        ×
}
```



void = nothing to return

“return” can be neglected in this case

Example#1 No parameter + No return data

```
using System;
public class Program
{
    public static void Main(){
        SayHi();
    }
    public static void SayHi(){
        Console.WriteLine("Hello");
        Console.WriteLine("Bonjour");
        Console.WriteLine("Sawasdee");
    }
}
```

Main method

SayHi method

calling

Output / Result

```
Hello
Bonjour
Sawasdee
```

How to get the output below?

```
Hello
Bonjour
Sawasdee
Hello
Bonjour
Sawasdee
Hello
Bonjour
Sawasdee
```

How to get the output below?

Output / Result

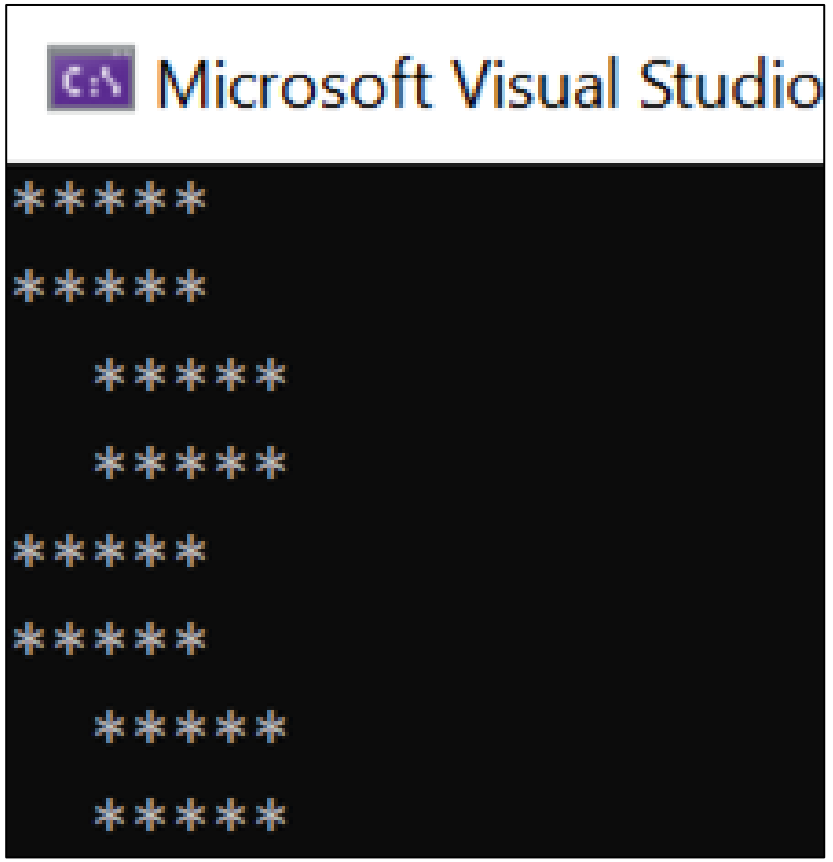
```
Hello
Bonjour
Sawasdee
Hello
Bonjour
Sawasdee
Hello
Bonjour
Sawasdee
```

```
using System;
public class Program
{
    public static void Main(){
        SayHi();
        SayHi();
        SayHi();
    }
    public static void SayHi(){
        Console.WriteLine("Hello");
        Console.WriteLine("Bonjour");
        Console.WriteLine("Sawasdee");
    }
}
```

Invoking / Calling method

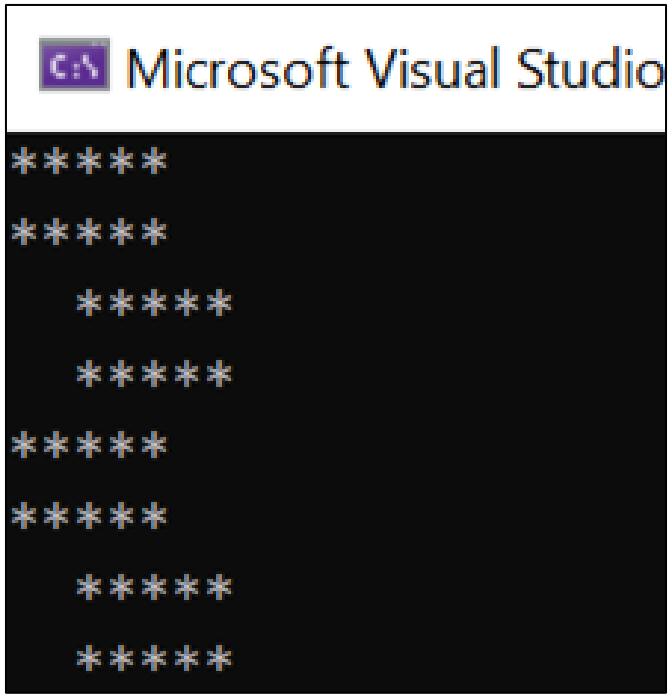
Exercise#1

- Create methods to display the result below



Exercise#1

Output / Result




```
using System;
namespace ConsoleAppCh8Ex1{
    public class StarPrint{
        public static void Main(string[] args){
            ShowPattern1();
            ShowPattern2();
            ?????
        }
        public static void ShowPattern1() {
            Console.WriteLine("*****");
            Console.WriteLine("*****");
        }

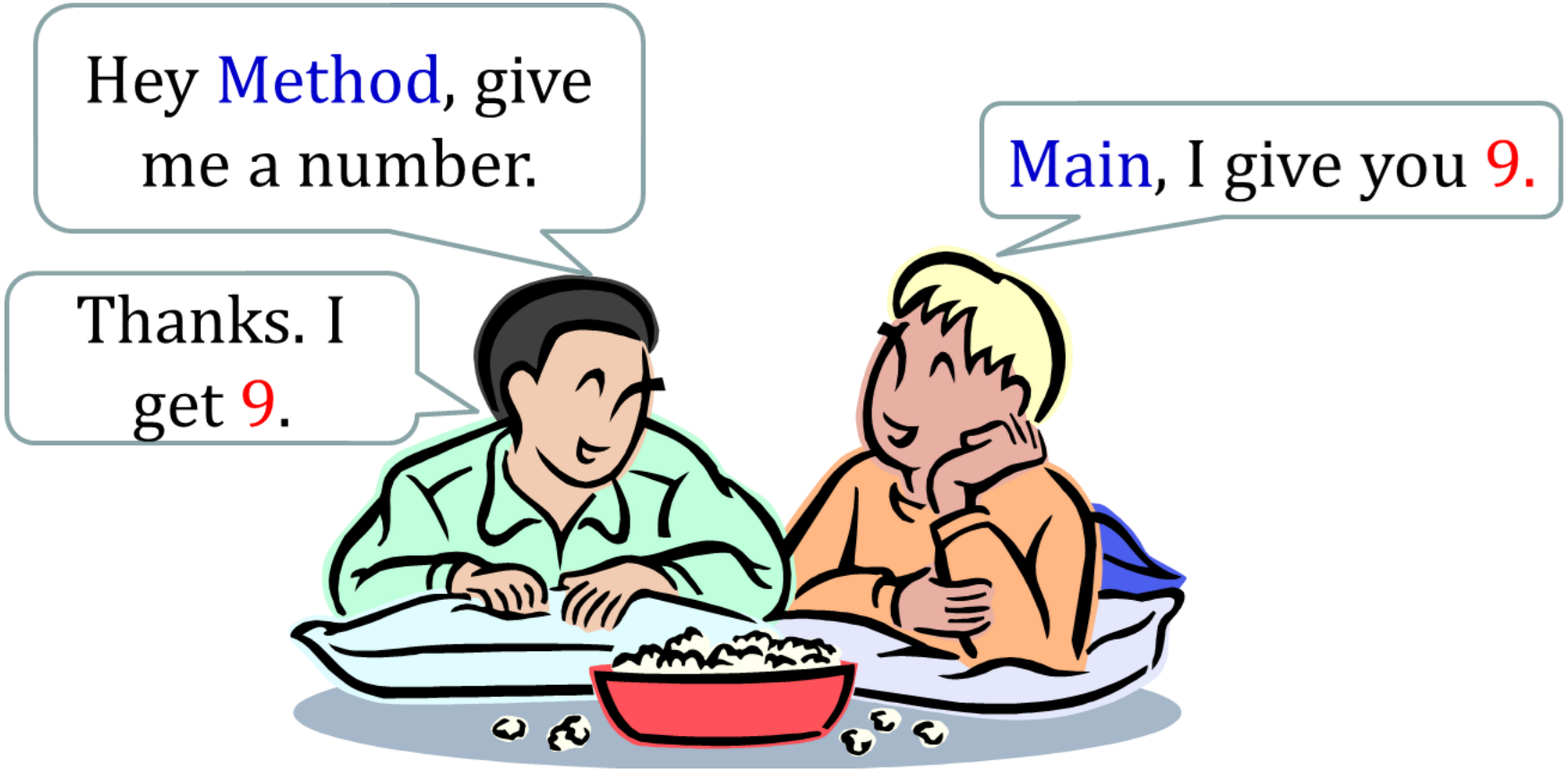
        public static void ShowPattern2() {
            Console.WriteLine("  *****");
            Console.WriteLine("  *****");
        }
    }
}
```

Missing code?

A. ShowPattern1();
   ShowPattern2();

B. ShowPattern2();
   ShowPattern1();

2. No parameter + Return data



2. No parameter + Return data

```
Modifier return_type method_name (parameters)
{
        statements;
        return variable or data;
}
```



```
public static return_type method_name (  )
{
        statements;
        return variable OR data ;
}
```

No parameter

Return data

Be careful!

```
public static return_type method_name ()
{
        statements;
        return variable OR data;
}
```



Must be the same data type

Must be a single value

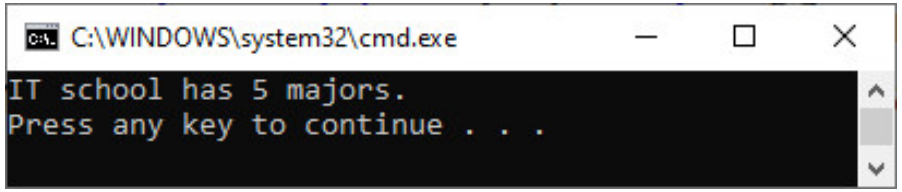
Example#2 No parameter + Return data

```
using System;
namespace ConsoleAppCh8Ex2{
    public class ITschool{
        public static void Main(string[] args){
            Console.WriteLine("IT school has {0} majors.", GetNumMajor());
        }
        public static int GetNumMajor() {
            int numMajor = 5;
            return numMajor;
        }
    }
}
```

call

return

Output / Result



3. Parameter(s) + No return data



3. Parameter(s) + No return data

```
Modifier return_type method_name (parameters)
{
        statements;
        return variable or data;
}
```



```
public static void method_name (parameters)
{
        statements;
        return ; //or nothing
}
```

Parameter(s)

No return data

Example#3 Parameter(s) + No return data

```
using System;
namespace ConsoleAppCh8Ex3
{
    public class Program{
        public static void Main(string[] args){
            ShowName("Lisa");
        }

        public static void ShowName(string name) {
            Console.WriteLine("Hello "+name);
        }
    }
}
```

Sending data
(Same data type)



Output / Result

```
Hello Lisa
```

Example#4 Parameter(s) + No return data

```
using System;
namespace ConsoleAppCh8Ex4
{
    public class Program{
        public static void Main(string[] args){
            ShowName("Lisa", 23 );
        }

        public static void ShowName(string name, int age) {
            Console.WriteLine("Hello "+name);
            Console.WriteLine("You are {0} years old.", age);
        }
    }
}
```



Two parameters of different types

Output / Result

```
Hello Lisa
You are 23 years old.
```

Exercise#2

- Create a method to display the results below

Expected Result1

```
Enter the first year: 2007
Enter the current year or last year: 2019
Working experience:  12 years
```

Expected Result2

```
Enter the first year: 2007
Enter the current year or last year: 2004
Invalid input!!!
```

Exercise#2 Solution

Expected Result1

```
Enter the first year: 2007
Enter the current year or last year: 2019
Working experience:  12 years
```

```
using System;
namespace ConsoleAppCh8Exercise2{
    public class Program{
        public static void Main(string[] args){
            Console.Write("Enter the first year: ");
            int tmpStartYear = Convert.ToInt32(Console.ReadLine());
            Console.Write("Enter the current year or last year: ");
            int tmpEndYear = int.Parse(Console.ReadLine());

            //Calling or Invoking method
            CalNumYear(  ?????  );
        }

        public static void CalNumYear(int startYear, int endYear) {
            if (  ?????  ) {
                Console.WriteLine("Invalid input!!!");
            }
            else {
                int numYear =  ?????
                Console.WriteLine("Working experience:  "+numYear+" years");
            }
        }
    }
}
```

Expected Result2

```
Enter the first year: 2007
Enter the current year or last year: 2004
Invalid input!!!
```

Missing 2 values for passing?

a) tmpStartYear, tmpEndYear
b) startYear, endYear

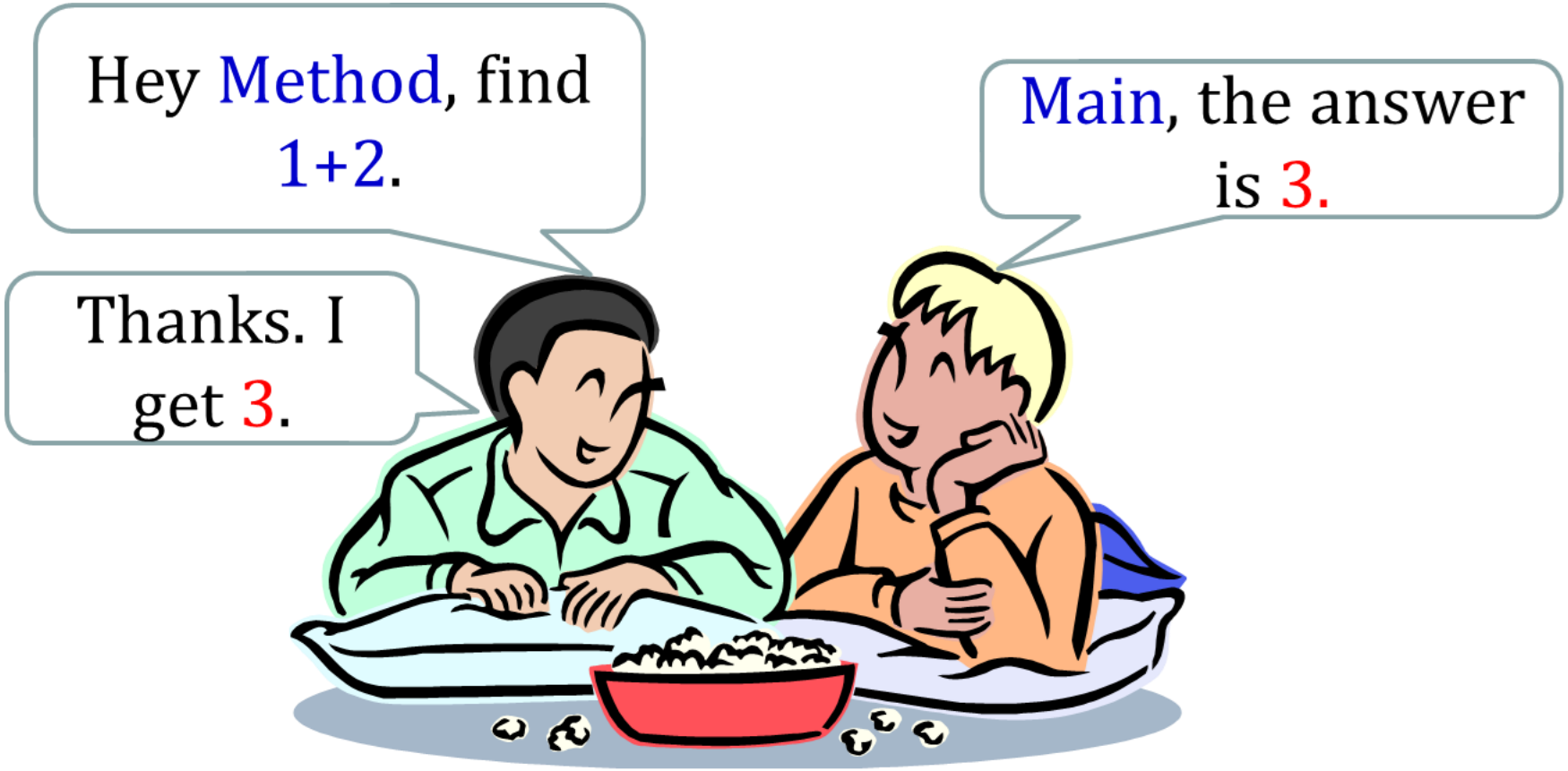
Missing condition?

a) startYear > endYear
b) startYear < endYear

Missing formula?

a) startYear – endYear;
b) endYear – startYear;

4. Parameter(s) + Return data



4. Parameter(s) + Return data

```
Modifier return_type method_name (parameters)
{
        statements;
        return variable or data;
}
```



```
public static return_type method_name (parameters)
{
        statements;
        return variable OR data ;
}
```

Parameter(s)

Return data

Example#5 Parameter(s) + Return data

```
using System;
namespace ConsoleAppCh8Ex5{
    public class Program{
        public static void Main(string[] args){
            Console.Write("Enter the first number: ");
            int tmpNum1 = Convert.ToInt32(Console.ReadLine());
            Console.Write("Enter the second number: ");
            int tmpNum2 = int.Parse(Console.ReadLine());

            Console.WriteLine("Sum = "+Sum(     ?????     ));
        }

        public static  ?  Sum(int num1, int num2){
            return num1 + num2;
        }
    }
}
```

Output / Result

```
Enter the first number: 9
Enter the second number: 4
Sum = 13
```

Exercise#3

Write a program to

1. Ask user for two numbers.
2. Send these two numbers to a method “FindMax()”.
3. In this method, compare two received values and return the maximum value to “Main()”.
4. In “Main()”, display this maximum value.

Expected Result1

```
Enter the first number: 5
Enter the second number: 9
Max = 9
```

Expected Result2

```
Enter the first number: 10
Enter the second number: 3
Max = 10
```



Exercise#3 Solution

```
using System;
namespace ConsoleAppCh8Exercise3{
    public class Program{
        public static void Main(string[] args){
            Console.Write("Enter the first number: ");
            int tmpNum1 = Convert.ToInt32(Console.ReadLine());
            Console.Write("Enter the second number: ");
            int tmpNum2 = int.Parse(Console.ReadLine());

            //Calling method (Parameter(s) + Return data)
            Console.WriteLine("Max = " + FindMax(tmpNum1, tmpNum2));
        }

        public static 3 FindMax( 1 ){
            int maxValue = 0;
            if (num1 >= num2) {
                maxValue = num1;
            }
            else {
                maxValue = num2;
            }
            return 2 ;
        }
    }
}
```

Expected Result1

```
Enter the first number: 5
Enter the second number: 9
Max = 9
```

Expected Result2

```
Enter the first number: 10
Enter the second number: 3
Max = 10
```

Summary

1. What is Method?

a group of statements for a specific purpose

2. Types of Method

- **By Source**
  1. Built-in / Standard C# Methods / Predefined method
  2. User-defined Methods

- **By Parameter and Return Data**
  1. **No** parameter + **No** return data
  2. **No** parameter + **Return** data
  3. Parameter(s) + **No** return data
  4. Parameter(s) + **Return** data

Q & A

